

Introduction to the course

Matteo Fasiolo

About me

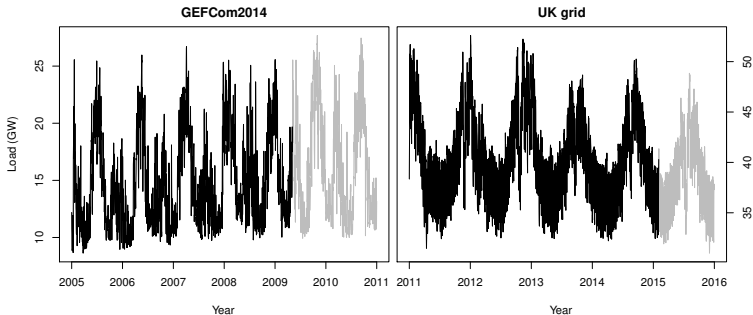
Matteo Fasiolo (matteo.fasiolo@bristol.ac.uk)

I have been programming mostly in R since 2006, I have authored several R packages:

- [mgcViz](#) for visualising regression models
- [qgam](#) for non-parametric quantile regression
- [mvnfast](#) for fast computation with the multivariate normal distribution

Until 2024 I taught a PhD course on [interfacing R with C++](#).

I do research on statistical modelling for energy applications.



When and Where

Type	Time	Location
Lectures	Mon, 11AM - 1PM	FRY BLDG 2.41
Labs	Tue, 11AM - 1PM	FRY BLDG LG.21 PC
Tutorials	Wed, 10AM - 12PM	FRY BLDG LG.21 PC

Hopefully this agrees with your Outlook calendars!

All material available [here](#):

https://github.com/mfasiolo/MATH10017_26

Structure and Key Objectives

The course is split in two parts:

- TB1, weeks 1 to 6: C Programming Language
- TB1, weeks 7 to 12: R Programming Language.

Upon completion of this unit you should be able to:

1. *Understand* the workflow of computer programming and appreciate computer as a data processing tool.
2. *Program, debug, document and test* basic algorithms in C and R.
3. *Decide* which programming language to use when faced with a computing task.

What are the Assessments?

- Written Exam (75%) at the end of TB1.
- Two Programming Courseworks (12.5% each).
 - CW1, Week 6 (26th October)
 - CW2, Week 11 (30th November)
- Non-assessed homework each week after lecture.
 - **Do not skip homework.**
 - These homework exercises are designed to build up skills required for the coursework.
 - Reuse some of the code you've written for these homework, to make your coursework easier.

Plagiarism and Collusion

- Read [this guideline](#) on Plagiarism very carefully.
- Read [this guideline](#) on Collusion very carefully.
- They are serious academic offences.
- Tips:
 - Never copy and paste from the internet/ChatGPT/Claude.
 - We encourage discussion but you should do your homework independently.

Policy of AI Use

Read the [Policy of AI Use](#).

In short, in the assessment, the use of AI is **prohibited**.

In the exam, **you will write code on paper**.

Why learning programming, if AI can do it for me?

Analogy: Portable calculators have been around since the 1970s.

They are better than any human at arithmetic.

But we haven't stopped teaching arithmetic in schools, why?

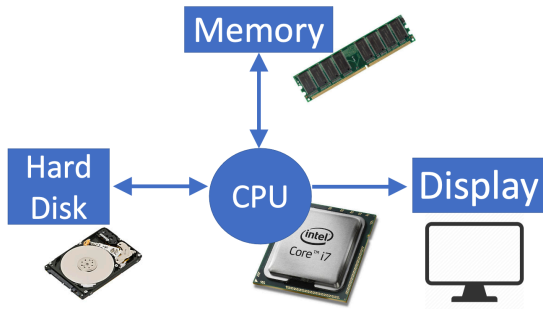
Because **goal of math education is cognitive development**, not just finding the final answer.

Coding develops problem-solving skills and logical thinking.

It also allows you to understand and judge what AI is doing.

Let's start: The foundation of computing

An Idealized Computer



von Neumann Architecture

- **Central Processing Unit (CPU)**

- Performs computational tasks.
- Controls Input/Output (IO) devices.
- Maintains data stored in the memory.

- **Memory** (random access memory, RAM)

- Stores program/data being used by CPU temporarily.

- **Input-ouput (IO) Devices**

- Hard disk
- Display
- Camera
- Touch Screen, etc.

What is Programming?

- Programming = writing a list of instructions to be executed on **the CPU**.
- The list of instructions is called the “code”.
- Programming is also called “coding”.
- Programmer is also called “coder”.
- The code is written in a **programming language**.

Low-level Programming Language

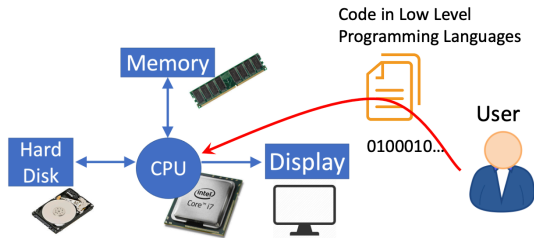


Figure 1: Low-level Programming Language Example

Coder can program in **machine or “native” code**.

Then the code can be **directly executed** on the CPU requiring no (or very little) translation.

Machine code (and its more human friendly variants) are referred to as “Low-Level Programming Languages”.

Advantages of Low-level Programming Languages:

- gives coder total control of hardware.
- can be efficient since it needs no translation.

Disadvantages of Low-level Programming Languages:

- can damage the hardware if the coder is not careful.
- is difficult to learn and read.
- only works on a specific cpu architecture (e.g. x86 or ARM).
Thus the code is not “portable”.

Example: Assembly is a type of low-level programming language.

It is basically machine code (01011...) in a readable format.

Assembly code printing a message "Hello, World" on display on x86 CPUs

```
        global    start
        section   .text
start:
        mov       rax, 0x02000004
        mov       rdi, 1
        mov       rsi, message
        mov       rdx, 13
        syscall
        mov       rax, 0x02000001
        xor       rdi, rdi
        syscall
        section   .data
message:
        db        "Hello, World", 10
```

[Source](#)

High-level Programming Languages

Coder can program in a more natural language, which is then **translated** into machine code.

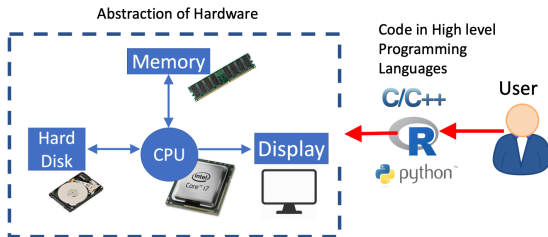
This translation process is called “compilation” and the software that performs this translation is called “**compiler**”.

This kind of languages are called “High-level Programming Languages”. For example,

- C, C++
- Python
- Java
- MATLAB
- R...

Since your code is not executed on the CPU “as is”, high-level language provides **an abstraction of hardware**.

Coder interfaces with this abstraction rather than directly program for the underlying CPU and hardware.



E.g.: you want to write code to show an image on the screen.

Your code does not need to specify the screen model.

Analogy: we drive cars using steering wheel, pedals and gearstick.

Advantages of abstractions:

- **Portability:**

- Driving skills are portable to different car models.
- The code is “portable”. The coder uses abstractions that shared across different CPU architectures.

- **Hides complexity:**

- You focus on driving, not on managing fuel-air mixture.
- The abstraction hides cumbersome details of hardware management, so you can focus on the computational task.

- **Safety:**

- You do not get a shock by handling ignition system.
- Abstractions prevents you to directly controlling the CPU.

However, **different programming languages** may provide **different levels of abstraction**.

Some high-level programming language allows you to directly manage computational resources to some extent.

Therefore, you can further categorize high-level programming languages into:

- “relatively low-level”, such as C/C++
- “relatively high-level”, such as Python, R and MATLAB

What is considered “high level” has evolved with time. See [here](#).

Disadvantages of High-level Programming Languages:

- Less efficient as the code requires “translation” before it can be executed on CPU.
- Cannot directly communicate with hardware: the coder has to interface with the abstraction.
- Abstractions come with resource overhead: they use memory and CPU.
- Tracking down hardware-related problems can be harder.

Intro to programming in C

Example C code for printing “Hello World!” on your screen:

```
// filename: main.c

#include <stdio.h>

int main(void){
    printf("Hello World!\n");
    return 0;
}
```

- //filename: main.c: **Comments**. Readable explanations of the source code. Ignored by compiler.
- #include <stdio.h>: **Preprocessing**. Instructs the compiler to perform tasks before compilation.
- int main(void){...}: **Function**. Contains list of statements.
- printf("Hello World!\n");: **Statement**. The actual command to be carried out by the CPU.

The C code in `main.c` is not directly executable on the CPU. First, it must be compiled, that is translated to machine code. Compilation (in the command line):

```
gcc main.c -o main
```

Execution (in the command line):

```
./main  
Hello World!
```

NOTE: you should type `./main`, not just `main`.

On Windows, type `.\main` instead.

Where `/` is slash and `\` is backslash.

The gcc compiler

```
gcc main.c -o main
```

- gcc: [GNU C Compiler](#). An [open-source](#) C programming compiler, available on most platforms.
- `main.c` C code file, as the input.
- `-o main` “main” as the output executable file (a file contains machine code, ready to be executed by CPU).

We give the executable no extension: on Windows gcc adds `.exe` by itself, so the same command works on Windows, Linux and Mac OS.

If you leave out `-o`, gcc calls the executable `a.out` (`a.exe` on Windows).

Example Code: R

How would this

```
#include <stdio.h>

int main(void){
    printf("Hello World!\n");
    return 0;
}
```

look like in R?

It's just one line of code:

```
print("Hello World!")
```

Much shorter/easier code and there is no explicit compilation!

Python code works similarly to R... **so why bother learning C?**

Why are we learning C?

C is closer to hardware than R or Python: you manage memory directly and see how data is laid out.

It helps you to understand how a computer works at a basic level.

That stays with you even after you switch to higher level languages.

R and Python code is implicitly calling lots of C code.

Often useful to re-implement R and Python code in C for efficiency.

What do you need to program in C?

To do C programming, you need two things at least:

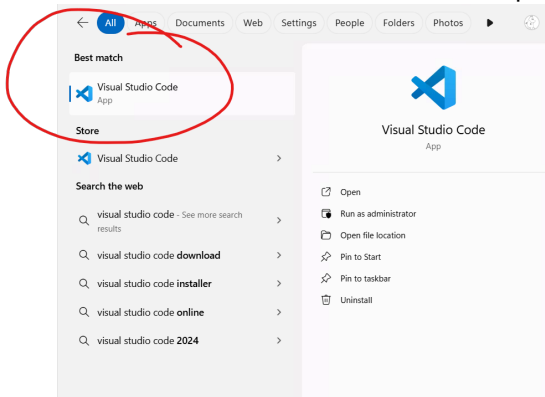
- **Text Editor:** As C code are text files, you need a text editor to write and organize these code files.
- **gcc:** The compiler to translate your C code into machine code. It comes with most Linux installations.

We call **development environment** collection of software to write, debug, document and test your code.

We recommend the [Visual Studio Code](#) integrated development environment (IDE).

For R we will use the RStudio IDE.

Visual Studio Code is installed in the computer lab:



We cannot help you to set up VSCode on your computer:

- If you want to use your computer, [set up VScode](#) before lab.
- Set up might be complicated on Windows and Macs.

Switch to VS Code

Open `Lab_main.c`

Conclusion

1. Computers and the Von Neumann architecture.
2. Programming = **writing a list of instructions to be executed on the CPU.**
3. There are two types of programming language: High level and low level.
 - Pros and cons of each.
4. Two things you need for C programming: Text editor + Compiler!

About `printf`: more details in this weeks tutorial!

Appendix: Resources about C Programming

Download the slides from Github before clicking the links!

1. [The C Programming Language](#), Brian Kernighan (the creator of C) and Dennis Ritchie.
 - 1.1 [An interview of Brian](#) on the history of C.
2. [Wikibooks: C Programming](#), for quick references.
3. [COMS10008: Imperative Programming](#)
 - 3.1 An excellent unit used to be taught by Dr. Ian Holyer in Computer Science department.
4. [CS50](#) is Harvard University's introductory course to computer science and the art of programming.
5. [C Programming under 4 hours](#) is a youtube video crash course designed for beginners.